I. forduló (Web verseny) 2026. november 6-8.

Egy régi, elhagyott bánya mélyén labirintusként kanyarognak a kamrák. A bányászok egykor sietve hagyták el őket, és a sötétben ma is csillogó aranyrögök várják, hogy valaki rájuk leljen. A bánya azonban nem kegyelmez, és egyetlen rossz lépés után már a fal állja az utat. A kincseket csak egy acélból kovácsolt felfedező, a kincskereső robot szerezheti meg. Sorról sorra hajtja végre a neki írt parancsokat: a sötétben csak azt érzékeli, ami közvetlenül előtte van, így tapogatózva halad előre, és ahol aranyra bukkan, ott kibányássza a rögöt. De a robot csak annyit tud, amennyit a programja elárul neki, és egyetlen elrontott utasítás elég ahhoz, hogy falnak menjen.

A feladatotok egy olyan webes játékalkalmazás elkészítése, amelyben egy speciális programnyelven (MineScriptben) megírt program alapján vezérelhető a robot a megadott labirintusban. Az alkalmazásnak lehetőséget kell biztosítania a program importálására, átszerkesztésére, szintaktikai ellenőrzésére és futtatására. A program futtatása közben az alkalmazás megjeleníti a labirintusban a robot mozgását, az általa felfedezett területet és a megtalált aranyat. A játékosok saját térképeket is készíthetnek, és versenyezhetnek egymással: egy ranglista mutatja, ki tudott egy adott térképen a programjával több aranyat, kevesebb lépésben összegyűjteni.

1. A térkép és a program formátuma

A térképek és a programok szöveges fájlként importálhatók és exportálhatók. A térképfájl egy térképet, a programfájl pedig egy MineScriptben megírt programkódot tartalmaz, ez irányítja a robotot.

Példa térképfájl:

```
3 15
....@..........
.##############.
.......@...>...
```

Példa programfájl:

```
set arany to isThere 2
print arany
```

A térképfájl első sorában a térkép kiterjedése található, egy szóközzel van elválasztva a két dimenzió. (Mindkét dimenzió legalább 1, maximum 15.) A példában 3 sorból és 15 oszlopból áll a térkép. A térképen szereplő szimbólumok jelentése:

- a . üres kamra, amiben épp nem található se a robot, se aranyrög,
- a `@` olyan kamra, amelyben aranyrög található,

- a `#` a fal, ami áthatolhatatlan kőzet,
- míg a `>`, `<`, `^`, `v` szimbólumok a robotot jelölik (a térképen pontosan egy robot van). A robot egy aranyrögöt nem tartalmazó kamrában áll, a szimbólum pedig a robot irányát is megadja, vagyis azt, hogy a négy égtáj közül merre néz: `>` kelet, `<` nyugat, `^` észak, `v` dél. A példában szereplő `>` tehát azt jelenti, hogy a robot épp kelet felé néz.

A programfájl a robotot irányító program utasításait tartalmazza. (A program lehet üres is.)

A példaprogram működése:

- Az `isThere x` függvény hívása logikai értékkel (`true`/`false`) tér vissza aszerint, hogy a robot előtti mezőn az `x` paraméternek megfelelő objektum található-e: `0` esetén üres kamra, `1` esetén fal, `2` esetén aranyrög.
- az `isThere 2` tehát megvizsgálja, hogy a robot előtti mezőn aranyrög van-e,
- ennek az eredményét az `arany` nevű változóba helyezi el,
- majd kiírja a konzolra az `arany` változó értékét.

Egy importált fájl formátuma hibás is lehet: ilyenkor az alkalmazásnak érthető hibaüzenettel kell jeleznie a hibát, és nem szabad a hibás tartalmat betöltenie vagy összeomlania.

2. A szöveges forráskód nyelvtana

Tekintsük az alábbi programot, ami addig vezeti a robotot előre, amíg az falba nem ütközik. Útközben a robot minden aranyrögöt kibányász, majd a program a futása végén kiírja, mennyi aranyrögöt bányászott ki.

```
set count to 0
set fal to isThere 1
set megy to not fal
while megy
  set arany to isThere 2
  if arany
    step forward 1
    mine here
    set count to count + 1
  else
    step forward 1
  set fal to isThere 1
  set megy to not fal
print count
```

Az EBNF (Extended Backus-Naur Form) egy formális jelölésrendszer, amellyel egy nyelv nyelvtana – vagyis az érvényes szerkezetek felépítése – tömören leírható: megadja, hogy az egyes elemek (tokenek, azaz a nyelv legkisebb önálló egységei, pl. kulcsszavak, azonosítók, számok) milyen sorrendben és milyen ismétlődéssel állhatnak össze.

Egy MineScript nyelven írt program a következő EBNF-jelöléssel definiálható:

```
programme     = { command }

command       = procedure | set
              | if | ifelse | while

procedure     = step | turn | mine | print
function      = isThere

step          = "step" "forward" simpleval
turn          = "turn" "by" simpleval
mine          = "mine" "here"
print         = "print" simpleval

isThere       = "isThere" simpleval

set           = "set" ident "to" calculatedval
if            = "if" simpleval block
ifelse        = "if" simpleval block "else" block
while         = "while" simpleval block

block         = { command }

calculatedval = simpleval
              | function
              | binop
              | unop

binop         = simpleval binoperator simpleval
unop          = unoperator simpleval

simpleval     = ident | number | boolean-lit

binoperator   = "+" | "-" | "*"
              | "=" | "<>" | "<" | "<=" | ">" | ">="
              | "and" | "or"
unoperator    = "not"

boolean-lit   = "true" | "false"
ident         = letter { letter | digit }
number        = digit { digit }

letter        = "a".."z" | "A".."Z"
digit         = "0".."9"
```

A kulcsszavak és az azonosítók is kis- és nagybetűérzékenyek: a kulcsszavakat pontosan a nyelvtanban megadott alakban kell írni (pl. `isThere`, nem `isthere`), az `arany` és az `Arany` pedig két különböző változó. A kódban szereplő egész számok 0 és 100 közöttiek lehetnek.

Az azonosítók (`ident`) nem lehetnek kulcsszavak, azaz az alábbi szavak egyike sem (semmilyen kis- vagy nagybetűs kombinációban): `and`, `by`, `else`, `false`, `forward`, `here`, `if`, `isThere`, `mine`, `not`, `or`, `print`, `set`, `step`, `to`, `true`, `turn`, `while`.

2.1 Soronkénti feldolgozás / tokenizálás

A forráskód tehát tokenekből épül fel, ezek a program legkisebb, önálló jelentéssel bíró egységei: a kulcsszavak (pl. `set`), a logikai értékek (pl. `true`), az operátorok (pl. `+`), az azonosítók (változónevek) és a számok. Amennyiben egy szöveges állományból szeretnénk feldolgozni a forráskódot, az EBNF-jelölés önmagában még nem elegendő, mert először a szövegben kell tudni helyesen beazonosítani a tokeneket. Szerencsére ez a folyamat nem bonyolult, és az alábbi algoritmus mentén elvégezhető:

1. **Sorokra bontás.** Minden sor egy utasítás, nincs külön pontosvessző vagy más elválasztó. A sor eleji behúzás adja a szintet, aminek egy szinten belül egységesnek kell lennie. A behúzás jelöli, mi tartozik egy `if`/`while` törzséhez; amint csökken, a törzs lezárul. Az `else` az `if` törzsével azonos behúzáson van. A behúzás csak szóközzel jelölhető (tab nem használható). A különböző blokkok esetén más és más lehet a behúzás mértéke (de a blokkon belül egységes).
2. **Tokenizálás.** A behúzás után a sort szóközök mentén daraboljuk tokenekre. Pontosan egy szóköz szerepel az egy soron belüli tokenek között, kivéve a sor elejét; ott - az előző pontban leírtak szerint - a behúzásnak megfelelően vannak a szóközök.
3. **A sor értelmezése.** Az első token adja meg, milyen utasításról van szó (`step`, `turn`, `mine`, `print`, `set`, `if`, `else`, `while`). Az utasítás kötelező kulcsszavai rögzítettek (`step forward …`, `turn by …`, `mine here`, `while …`, `set … to …`), a nem rögzített tokenek pedig `simpleval`-ok.
4. Nincs beágyazott kifejezés, nincs rekurzió: minden `simpleval` egyetlen token. Ezért elég a sorolvasás és a szódarabolás.
5. A forráskód nem tartalmaz utasítás nélküli - azaz üres - sorokat.

Mivel a programkód a felületen szabadon szerkeszthető, illetve fájlból importálható, nem feltételezhető, hogy a program szintaktikailag helyes. Az alkalmazásnak ezért futtatás előtt ellenőriznie kell, hogy a program megfelel-e a fenti nyelvtannak és feldolgozási szabályoknak (pl. hibás vagy blokkon belül nem egységes behúzás, tab a behúzásban, üres sor, ismeretlen utasítás stb.). Szintaktikailag hibás program nem futtatható: a hibás sor megjelölésével kell jelezni.

2.2 Szemantikai szabályok

Míg a nyelvtan (EBNF) csak a program struktúráját írja le, addig az ún. szemantikai szabályok a jelentését és a végrehajtás közbeni elvárásokat rögzítik. Azaz a szemantikai szabályok azt határozzák meg, hogy a szintaktikailag helyes program hogyan viselkedik futás közben: mit jelentenek az egyes utasítások, milyen típusokkal dolgozhatunk, és mikor számít egy program hibásnak.

Számos szemantikai szabály tartozik a MineScript nyelvhez, amelyek rendeltetésük szerint csoportosíthatóak.

2.2.1 Kulcsszavak működése

- **Aritmetika** (`+ - *`): egész operandusok, egész eredmény.
- **Összehasonlítás** (`< <= > >=`): egész operandusok, logikai eredmény.
- **Egyenlőségvizsgálat** (`= <>`): egész és logikai operandusokra is működik, de mindkét operandusnak azonos típusúnak kell lennie (mindkettő egész vagy mindkettő logikai). Az eredmény logikai.
- **Logika** (`and`, `or`, `not`): logikai operandusok, logikai eredmény.
- `set`: a `set x to ertek` utasítás először megpróbálja kiértékelni a `to` utáni kifejezést, majd siker esetén az eredményt az `x` nevű változóba írja. Ha az `x` még nem létezik, az első értékadásnál jön létre, és az érték típusa lesz a változó végleges típusa. Ha már létezik, az új értéknek azonos típusúnak kell lennie a korábbival. A `to` utáni rész (`calculatedval`) kiértékelése balról jobbra történik: ha művelet áll benne, annak operandusait egyenként olvassa be, és mindegyiket azonnal ellenőrzi, hogy a típusa megfelel-e a műveletnek, mielőtt a következőre lépne. Az első hibánál a kiértékelés megáll. Ha az eredmény már az első operandusból következne (pl. `true or c`), a másodikat akkor is beolvassa. Az = és a <> első operandusa bármilyen típusú lehet, náluk a két típus egyezését a második operandus beolvasása után ellenőrzi.
- `if`**:** a feltétel szerint a `then`- vagy (ha van) az `else`-ág fut.
- `while`**:** a törzs addig ismétlődik, amíg a logikai feltétel igaz.

2.2.2 A nyelv alapszabályai

- **Két típus:** egész (integer) és logikai (boolean), köztük nincs implicit konverzió.
- **Nincs változódeklaráció:** a változó az első `set`-kor jön létre, és a program végrehajtásának végéig él.
- **Típusrögzítés:** a változó típusa az első értékadáskor véglegesen rögzül, később nem változtatható.
- **Feltétel:** az `if` és a `while` feltétele logikai kell legyen;
- **Minden típusellenőrzés futásidőben** történik, nincs fordítás előtti ellenőrzés.
- **Végrehajtás:** a sorok fentről lefelé, szekvenciálisan futnak; a blokk az azonos mélységbe behúzott sorokból áll.

2.2.3 Robot API

A robot vezérlése a robot API meghívásán keresztül lehetséges. Az egyes API-végpontok natívan a MineScript nyelv részét alkotják.

- `step forward n`: a robot egyetlen hívással n egységet halad előre (n nemnegatív egész), közben a köztes mezőket is érintve. Ha olyan mezőt érint, ahol aranyrög található, ott hagyja.

- `turn by n`: az `n` értéke nemnegatív. A robot n negyedfordulatot lép balra (azaz az óramutató járásával ellentétes irányba). 0 esetén a robot iránya nem változik.
- `mine here`: a robot a mezőn, amelyen áll, kibányássza az aranyrögöt, ezután már ezen a mezőn nem lesz több bányászható arany;
- `isThere w`: a robot előtt lévő mező fajtáját ellenőrzi: a `w` paraméter (`0` = üres kamra, `1` = fal, `2` = aranyrög) alapján logikai értékkel tér vissza (`true`/`false`). Amennyiben a robot vizsgálódása a térkép szélét meghaladja, azt falnak tekinti és ekképpen tér vissza a megfelelő isThere hívásra.
- `print x`: kiírja az egész vagy logikai értéket a konzolra.

(Az `n`, `w` és `x` paraméter - a nyelvtan alapján - mind `simpleval` kifejezés.)

2.2.4 Futásidejű hibák

A program **végrehajtása során** az alábbi hibák következhetnek be, amiket az alkalmazásotoknak kezelnie szükséges:

- **Típust váltó értékadás (**hibakód**:** `E_TYPE_MISMATCH`): egy már rögzített típusú változóhoz eltérő típusú értéket rendelsz.
- **Nem-inicializált változó olvasása (**hibakód**:** `E_UNINITIALIZED`): érték hozzárendelése előtt hivatkozol egy változóra.
- **Nem-logikai feltétel (**hibakód**:** `E_NON_BOOL_CONDITION`): az `if` vagy `while` feltétele nem logikai (pl. egész).
- **Rossz operandustípus (**hibakód**:** `E_BAD_OPERAND`): aritmetikai (`+ - *`) vagy összehasonlító (`< <= > >=`) művelet nem egész operandussal, logikai művelet (`and, or, not`) nem logikai operandussal, illetve egyenlőségvizsgálat (`= <>`) két különböző típusú operandussal.
- **Falra lépés vagy lelépés a térképről (**hibakód**:** `E_BEDROCK_HIT`): `step forward` a soron következő lépésnél falra vezetne vagy lelépne a térképről.
- **Bányászás arany nélkül (**hibakód**:** `E_NO_GOLD`): `mine here` olyan mezőn, ahol nincs vagy nincs már arany.
- **Érvénytelen paraméter (hibakód:** `E_BAD_ARG`**)**: egy API-hívás a megengedett tartományon kívüli paraméterrel vagy rossz típusú paraméterrel hívódik meg. Pl. `isThere w`, ahol `w` nem a 0, 1 vagy 2 értékek egyike, vagy `turn by true`.

Ezen hibák előfordulása esetén a robotprogram végrehajtása befejeződik, a hibát pedig a felületen is jelezni kell.

3. A játék működése

A megvalósítandó alkalmazás tehát egy böngészőben futó webes játék. Az alkalmazásban a következő funkciók érhetőek el:

- bejelentkezés és felhasználókezelés,

- térképszerkesztő, térképek importálása és exportálása,
- a program importálása és szerkesztése szöveges kódszerkesztőben,
- a program futtatása,
- ranglista.

Amennyiben elkezdődik a program futtatása, a térkép és a robot helyzete alaphelyzetbe kerül. Az alábbi alfejezetek az egyes funkciókkal szemben támasztott alapkövetelményeket írják le. Ezeken túl a funkciók tervezése és megvalósítása – a felület kialakításától a felhasználói folyamatokon át az adatok tárolásának módjáig – rátok van bízva.

3.1 Bejelentkezés és felhasználókezelés

A ranglista miatt minden futtatás eredményét egy felhasználóhoz kell kötni, ezért az alkalmazás funkciói bejelentkezés után érhetőek el. A felhasználókezelés módját a csapat választja meg: lehet pl. szabad regisztráció, vagy pl. az, hogy a felhasználókat egy adminisztrátor hozza létre.

3.2 A térképszerkesztő

A térképszerkesztővel új térkép hozható létre, illetve egy meglévő térkép módosítható: megadható a térkép mérete, és az egyes mezőkre üres kamra, aranyrög, fal vagy a robot (a kezdőirányával együtt) helyezhető. A szerkesztőnek ügyelnie kell arra, hogy csak az 1. fejezetben leírt szabályoknak megfelelő térkép legyen menthető (pl. mindkét dimenzió legalább 1, legfeljebb 15, a térképen pontosan egy robot van, és az aranyrögöt nem tartalmazó kamrában áll). A térképek az 1. fejezetben leírt formátumban importálhatók fájlból és exportálhatók fájlba. Az, hogy a térképek felhasználónként külön vagy minden felhasználó számára közösen érhetőek-e el, a csapat döntése.

3.3 A programkód szerkesztése

A program szöveges kódszerkesztőben írható és módosítható, illetve az 2. fejezetben leírt formátumú programfájlból importálható és fájlba exportálható. A program szerkesztés közben még nem feltétlenül helyes, csak a futtatás előtt kell azzá válnia.

3.4 A program futtatása

Futtatni csak a 2.1 fejezet szerint szintaktikailag helyes programot lehet; hibás program esetén a felület jelzi a hibát és a hibás sort. Indításkor a térkép és a robot alaphelyzetbe kerül, majd a program lassítva fut, hogy a robot mozgása jól követhető legyen. Futás közben látszódjon az éppen végrehajtott sor, a print kimenete pedig egy konzolon jelenjen meg.

A futtatás közben az alkalmazás megjeleníti a térképet, és rajta a robot aktuális helyzetét és irányát. Mutatja a már felfedezett mezőket – azokat, amelyeken a robot már járt, vagy amelyekre meghívódott az isThere függvény – megkülönböztetve a még feltérképezetlen területektől. A már kibányászott aranyrögök helyét nem szükséges külön jelölni a térképen.

A program futása közben az alkalmazás számolja a feldolgozott programsorokat: minden végrehajtott sor egynek számít. Ciklusnál ez azt jelenti, hogy a törzs sorai minden lefutáskor újra

számítanak, a `while` sora pedig minden feltétel-kiértékelésnél, a visszaugrásnál is számít. Az `else` sora csak akkor számít, ha az else ág lefut, vagyis ha az `if` feltétele false volt.

A program futása során legfeljebb 1000 programsor dolgozható fel. Ha a program az 1000. feldolgozott programsor után újabb sor feldolgozására készülne, a futás megszakad. Ilyenkor az alkalmazásnak a felületen az `INSTRUCTION_COUNT_LIMIT_EXCEEDED` hibát kell jeleznie.

A futás véget érhet a program végén hiba nélkül vagy futásidejű hibával (ld. 2.2.4) vagy a futási limit túllépésével. Hiba esetén a felület mutassa a hibakódot és a hibás sort.
A futás közben az alkalmazás kijelzi a kibányászott aranyrögök számát és a lépésszámot; ez kerül a ranglistára (ld. 3.5). A futás leállítható.

3.5 A ranglista

Minden programfuttatás végén az alkalmazás rögzíti, hogy az adott felhasználó a betöltött programmal az adott térképen hány aranyrögöt bányászott ki, és ehhez hány lépésre volt szüksége. Lépésnek a futás során feldolgozott programsorok számát tekintjük (ld. 3.4). A ranglista térképenként mutatja az eredményeket: előrébb az áll, aki több aranyat szedett össze, azonos aranymennyiség esetén pedig az, akinek ehhez kevesebb lépés kellett.

4. Beadandó

A beadott anyag tartalmazza az alkalmazás működésének rövid leírását, amelyből egyértelműen kiderül:

- melyik funkciót hogyan lehet elérni;
- mely részek nem készültek el vagy kimaradtak;
- melyek számítanak plusz szolgáltatásnak az alapfeladathoz képest;
- milyen problémák merültek fel a fejlesztés során, és azokat hogyan sikerült megoldani.

A beadandó értékelésénél fontos szempontok:

- a hibátlan működés,
- az áttekinthető és esztétikus megjelenés,
- az akadálymentes használhatóság,
- valamint a jól olvasható, rendezett forráskód.

További előnyt jelent, ha a pályázók ötleteket vagy javaslatokat is megfogalmaznak a játék továbbfejlesztésére.

A program bemutatásához készüljön el egy olyan verzió, amelyben az alkalmazás futásakor rendelkezésre áll már egy játékkörnyezet.

Jó munkát kíván a versenybizottság!